ABDELMALEK ESSAADI UNIVERSITY
National School of Applied Sciences of Al Hoceima
Department of Mathematics and Computer Science
TDIA 2

Project Report

Darija AI Translator

Design and Development of a Full-Stack Flask-Based Application for Moroccan Darija Translation Using Deep Learning Models

Supervised by: Prof. Cherradi Mohammed
Prepared by: Essouiri Aya & Taouni Ouiam
Date: May 30, 2026

Academic Year: 2025 / 2026

Table of Contents

Abstract

This paper presents the design and development of a full-stack Flask-based application for translating Moroccan Darija into English using deep learning models. The system supports three input modalities: text, voice, and images. For text translation, we fine-tuned two transformer-based models from the Helsinki-NLP family: opus-mt-mul-en [6] for Latin-script Darija and opus-mt-ar-en for Arabic-script Darija. These models were trained on a combination of the MADAR parallel corpus and the DODa (Darija Open Dataset) [1], containing over 86,000 translated sentences. For comparative analysis, we also implemented a custom LSTM-based sequence-to-sequence model from scratch using PyTorch, following established seq2seq architectures [5]. The transformer-based models significantly outperformed the LSTM baseline by achieving lower validation loss and generating more fluent and contextually accurate translations. The application integrates Whisper for speech recognition and EasyOCR for optical character recognition, enabling comprehensive multi-modal input processing. Text preprocessing techniques [3] and visualization methods [2, 4] were employed during data analysis. The system also includes user authentication and translation history storage, making it a complete and practical solution for Darija-English translation.

Keywords: Darija translation; Neural Machine Translation; Transformer; LSTM; Flask; Speech recognition; OCR; Multi-modal AI

1 Introduction

Machine translation for low-resource dialects remains one of the most challenging tasks in natural language processing. While significant progress has been made for high-resource languages such as English, French, and Chinese, dialectals especially those without standardized writing systems continue to be excluded from the benefits of modern neural machine translation.

1.1 Context

Moroccan Darija is a spoken Arabic dialect used daily by millions of Moroccans. Unlike Modern Standard Arabic (MSA), Darija has no standardized orthography and has significant variations in spelling, vocabulary, and grammar. It combines influences from Arabic, Amazigh (Berber), French, and Spanish. Despite being the primary language of communication in Morocco, Darija remains largely underrepresented in natural language processing (NLP) resources. Most machine translation systems focus on MSA or other major languages, leaving Darija speakers with limited access to AI-powered translation tools. In recent years, the field of machine translation has witnessed remarkable advances driven by deep learning architectures, particularly transformer-based models. However, these advances have largely bypassed low-resource dialects such as Darija.

1.2 Problem Statement

Although various machine translation methods have been proposed for Arabic dialects, most are constrained to Modern Standard Arabic or major dialects such as Egyptian or Levantine. Moroccan Darija remains severely under-resourced. The existing solutions face several critical limitations. First, there is limited availability of parallel corpora specifically designed for Darija-English translation, making data-driven approaches difficult. Second, the dual-script nature of Darija which is written in both Latin and Arabic scripts requires separate handling because transliteration inconsistencies introduce noise into any single model approach. Third, the high variability in spelling and vocabulary due to the lack of standardization further complicates model training and evaluation. Fourth, even when models exist, they are rarely packaged into accessible translation tools for non-technical users, limiting real-world adoption. As emphasized by recent NLP literature, dialectal Arabic translation is a highly complex problem for which no satisfying solution currently exists for Moroccan Darija. This constitutes a significant limitation, as several important applications in communication, education, and content localization require Darija translation capabilities.

1.3 Contributions

To address these problems, this paper presents a complete multi-modal Darija-English translation platform with the following contributions. First, we provide a fine-tuned transformer-based translation model for Latin-script Darija using the Helsinki-NLP/opus-mt-mul-en architecture [6]. Second, we provide a fine-tuned transformer-based translation model for Arabic-script Darija using the Helsinki-NLP/opus-mt-ar-en architecture. Third, we implement a custom LSTM-based Seq2Seq model from scratch in PyTorch [5] to serve as a baseline for comparative analysis. Fourth, we develop a comprehensive dataset preparation pipeline that integrates the MADAR and DODa corpora [1]. Fifth, we build a full-stack Flask web application that supports text, voice (using Whisper), and image (using EasyOCR) inputs. Sixth, we incorporate user authentication and a translation history management system to enable personalized usage.

Our approach differs from existing methods in two important ways. First, it handles both Latin and Arabic Darija scripts through separate specialized models rather than forcing a single model to handle inconsistent transliterations. Second, it provides a practical multi-modal interface that is accessible to non-technical users, thereby bridging the gap between research-grade models and everyday usability.

2 Literature Review

2.1 Arabic Dialect Translation

Machine translation for Arabic dialects has gained increasing attention in recent years. For Moroccan Darija specifically, research remains scarce despite the dialect being spoken by over 30 million people.

2.2 Existing Datasets

The MADAR (Multi-Arabic Dialect Applications and Resources) project, released by Carnegie Mellon University Qatar and NYU Abu Dhabi, provides parallel sentences across 25 Arab cities. It includes two main subsets: Corpus-6 with 12,000 sentences translated into five major dialects plus MSA, and Corpus-26 with 2,000 sentences translated into 25 dialects. While valuable, the Moroccan representation (Rabat and Fes dialects) is limited.

The DODa (Darija Open Dataset) represents the largest open-source collaborative project for Moroccan Darija. It includes over 86,000 translated sentences across multiple CSV files organized into semantic and syntactic categories. A key feature of DODa is its capture of spelling variations, with the same word appearing in multiple Latin and Arabic forms. This dataset was specifically built for NLP purposes and continues to be improved through community contributions.

2.3 Translation Approaches

For sequence-to-sequence translation, two dominant architectures have emerged. LSTM-based encoder-decoder models have been widely used for machine translation, compressing input sequences into fixed-length context vectors. However, these models struggle with long-range dependencies. Transformer models address this limitation through self-attention mechanisms, processing entire sequences in parallel and capturing contextual relationships regardless of distance.

For low-resource dialects like Darija, transfer learning through fine-tuning pretrained models has proven effective. The Helsinki-NLP opus-mt models provide multilingual and Arabic-specific translation baselines that can be adapted to dialectal variants.

2.4 Research Gap

While previous work has established datasets and baseline models for Arabic dialect translation, several gaps remain. First, no dedicated solution exists for Moroccan Darija that handles both Latin and Arabic scripts. Second, most research focuses on model architecture without considering practical deployment. Third, multi-modal input (voice and image) has not been integrated with Darija translation. Our work addresses these gaps by providing a complete, deployable system with comparative analysis of LSTM and transformer approaches.

3 Methodology

3.1 Workflow Summary

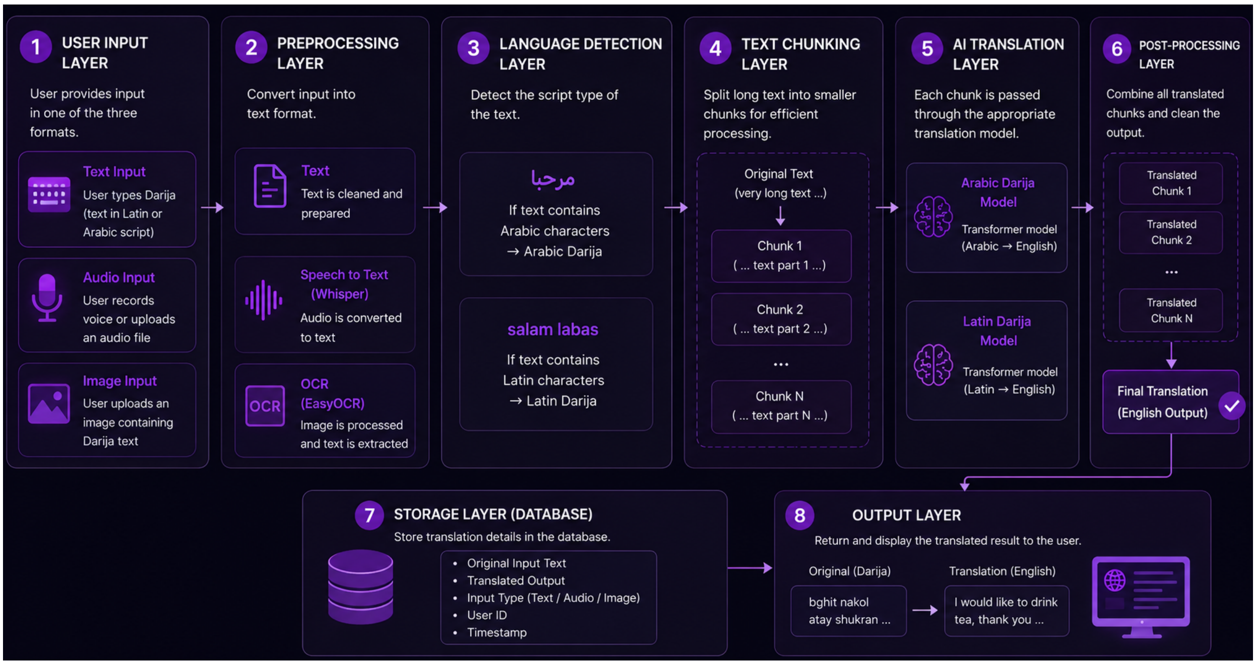

Figure 1: Complete translation workflow pipeline

3.2 Data Collection and Dataset Preparation

3.2.1 Data Sources

For this deep learning project, we used five sources. The first four were obtained from Hugging Face, while the last one was collected from a GitHub repository. However, in the implementation code, the dataset appears to be loaded directly from compressed ZIP files. That is because for the Hugging Face dataset we had an issue where we lost our code and later on the Hugging Face dataset was no more accessible. The dataset that was used is called MADAR parallel Corpora dataset, a multilingual Arabic dialect dataset released by Carnegie Mellon University Qatar and NYU Abu Dhabi.

The MADAR dataset contains parallel sentences across multiple Arabic dialects. For our project, we chose the Moroccan dialects (specifically Rabat and Fes dialect). The MADAR corpus is primarily built from the Basic Traveling Expression Corpus (BTEC) and is designed for dialectal Arabic machine translation research. It includes two main subsets: - Corpus-6: 12,000 sentences translated into five major dialect cities plus MSA - Corpus-26: 2,000 sentences translated into 25 Arab city dialects plus MSA

Each file contains aligned sentence pairs, making it suitable for supervised translation tasks.

As for the second dataset, it is from a GitHub repository called DODa (Darija Open Dataset). It is arguably the largest open source collaborative project and is built specifically for Natural Language Processing purposes.

This dataset is very versatile and rich. It includes 86,000 entries, and DODa takes into account the diversity of the Darija spelling, so we have the same words spelled in different ways. The entries are written in both Latin and Arabic with the English translation.

The dataset includes more than 86,000 translated sentences and is structured across multiple CSV files organized into different repositories and categories. These files contain various types of linguistic data, including words, verbs, sentences, names, and other lexical and syntactic elements.

DODa is designed to support both semantic and syntactic analysis. It provides multiple representations of the same word, including different spellings in Latin and Arabic scripts. The dataset also includes verb-to-noun transformations, masculine and feminine forms, and conjugations of hundreds of verbs across different tenses.

A key feature of DODa is its ability to capture the diversity of Darija spelling variations, reflecting how the language is used in real-world contexts. Entries may appear in multiple forms, which makes the dataset particularly suitable for robust NLP applications.

The dataset is widely used for research in machine translation, language modeling, and dialect understanding. It aims to become a reference dataset for Moroccan Arabic NLP and is continuously improved through community contributions.

3.2.2 Data Acquisition and Integration

As said before, the first dataset was already ready and just turned into a dataframe.

For the second dataset, it is way more complicated. This dataset is organized into multiple repositories:

1. Semantic categories This folder contains full sentences grouped by meaning, so each category is stored in a csv file with their translation. It helps the models learn how meaning changes across dialects, not just word-to-word translation.

2. Syntactic categories This folder focuses on sentence structure. It includes sentences designed to test grammatical patterns and variations in sentence structure across dialects. It helps the model learn how grammar changes across Arabic dialects, not just vocabulary.

3. X-tra This folder contains extra/additional data. It has supplementary sentences. It increases dataset diversity and helps improve model generalization.

After this, once we inspected the different folders, we looked at the structure of each one and noticed that we had words written in different ways with the same translation. So we came up with a code to keep 2 columns by adding the different ways as new rows and give them the same translation.

Lastly, we concatenated both data sources after ensuring that they have the same structure, and we also separated the Latin darija from the darija written in Arabic, so we ended up with 2 dataframes.

After that, we stored the datasets into two formats: parquet and csv.

3.2.3 Data Cleaning and Preprocessing

After storing the final datasets and making sure they were exactly how we wanted, the next step consisted of cleaning, preprocessing, and analyzing the data before training. The datasets were stored in Google Drive and loaded into Google Colab using the pandas library.

As explained before, we have two datasets: the Latin Darija dataset and the Arabic Darija dataset. Each dataset contains Darija sentences alongside their corresponding English translations.

To improve data quality and ensure better model performance, several preprocessing operations were applied. First, missing values were removed from both datasets to eliminate incomplete sentence pairs.

Rows containing empty English or Darija fields were discarded because such entries could negatively affect the learning process. Duplicate rows were also removed in order to avoid training bias caused by repeated examples. This helped produce cleaner and more reliable datasets for the translation models.

After removing invalid entries, text normalization was performed. A custom text cleaning function was created using Python and regular expressions. The preprocessing pipeline converted all text into lowercase format to reduce vocabulary inconsistency. Extra spaces and formatting inconsistencies were removed to standardize sentence structure across the datasets. This normalization process is important because deep learning models perform better when the training data follows a consistent format. However, numerical characters were preserved because Darija is frequently written using numerals as phonetic representations of Arabic sounds.

Once the cleaning phase was completed, the datasets were inspected using Pandas functions to verify the structure of the data. This allowed checking the number of samples, column types, and the absence of remaining null values.

3.2.4 Dataset Visualization

To better understand the content of the datasets, word frequency analysis was performed. The frequency of words appearing in both English and Darija sentences was calculated in order to identify the most commonly used vocabulary. The fifteen most frequent words in each dataset were visualized using bar charts generated with Matplotlib. This analysis provided insights into the linguistic distribution of the corpus and helped identify dominant vocabulary patterns in both Latin and Arabic Darija.

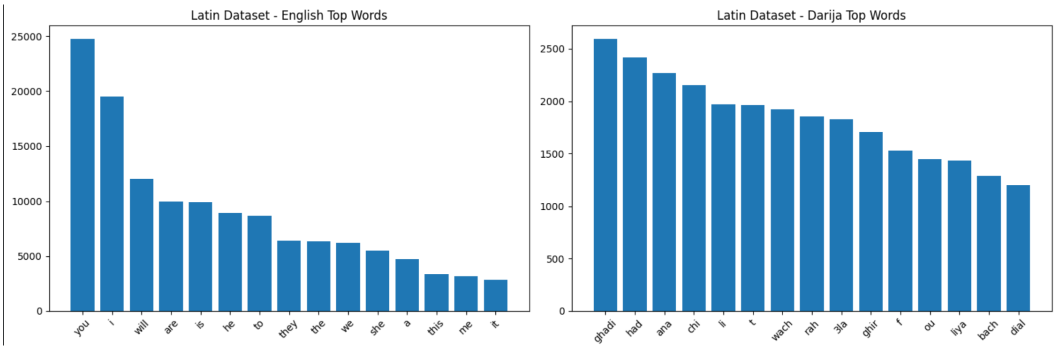

Figure 2: Common words frequency distribution

In addition to word frequency analysis, sentence length analysis was conducted to study the distribution of sentence sizes within the datasets. The number of words contained in each sentence was calculated and visualized using histograms. This analysis was important because sentence length directly affects sequence padding and tokenizer configuration during model training. Understanding the distribution of short and long sentences also provided insight into the overall complexity of the dataset.

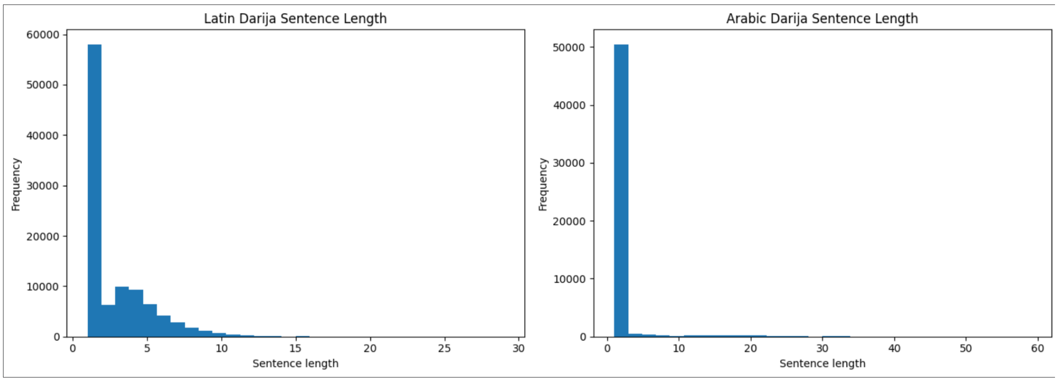

Figure 3: Sentence length distribution

WordCloud visualizations were also generated to provide a graphical representation of the most frequently used words. Separate WordClouds were created for English, Latin Darija, and Arabic Darija text. The Latin Darija dataset could be visualized directly.

However, Arabic text required additional preprocessing because Arabic characters are written from right to left and require proper reshaping for visualization. To solve this issue, additional libraries such as arabic-reshaper and python-bidi were installed. An Arabic-compatible font (DejaVuSans.ttf) was also used to correctly render Arabic text inside the WordClouds. These visualizations helped provide a clearer understanding of the lexical content and dominant terms present in the datasets.

Figure 4: WordCloud visualization results

Another important step in the exploratory analysis phase was vocabulary size computation. The total number of unique words was calculated for both the Latin Darija and Arabic Darija datasets. Vocabulary size is a significant factor in natural language processing because it influences tokenizer construction, embedding dimensions, and model complexity. A larger vocabulary generally indicates greater linguistic diversity but may also increase training complexity.

Finally, after completing preprocessing and exploratory analysis, the cleaned datasets were saved again in both CSV and Parquet formats. Saving the data in CSV format ensured readability and compatibility, while the Parquet format provided faster loading times and more efficient storage for large-scale deep learning experiments.

3.2.5 Dataset Splitting

Each dataset was divided into three subsets:

- Training: 70% of data
- Validation: 15% of data
- Testing: 15% of data

The final datasets were stored in both CSV and Parquet formats.

3.3 Vocabulary and Tokenization Analysis

Exploratory data analysis was performed to understand dataset characteristics. Word frequency analysis identified the fifteen most frequent words in each dataset. Sentence length analysis was conducted to determine sequence padding requirements. Vocabulary size was computed for both Latin and Arabic Darija datasets.

For text visualization, WordClouds were generated. Arabic text required additional preprocessing using arabic-reshaper and python-bidi libraries with an Arabic-compatible font (DejaVuSans.ttf).

3.4 LSTM-Based Sequence-to-Sequence Model

A custom neural machine translation system was implemented from scratch using PyTorch to serve as a baseline. The goal of this model was to manually learn the mapping between Darija sentences and English translations without relying on pretrained architectures, in order to better understand the underlying mechanism of machine translation systems.

3.4.1 Vocabulary Construction

A custom vocabulary was built using word frequency analysis with the Counter class. The following special tokens were defined:

- `<PAD>`: padding sequences to equal length
- `<SOS>`: start of sentence marker
- `<EOS>`: end of sentence marker
- `<UNK>`: unknown words

The vocabulary was limited to 20,000 most frequent words.

3.4.2 Tokenization

Sentences were tokenized by splitting into words and mapping each word to its vocabulary index. Sentences were truncated to a maximum length of 30 tokens. A custom collate function was implemented for dynamic padding within batches.

3.4.3 Model Architecture

The model follows a classical encoder-decoder LSTM architecture:

- **Encoder**: Embedding layer converting token indices to dense vectors, followed by an LSTM layer producing hidden and cell states as compressed sentence representations.
- **Decoder**: Embedding layer for target tokens, LSTM layer generating output step-by-step, and a fully connected layer mapping hidden states to vocabulary probabilities.

Teacher forcing was used during training, where the correct previous word is sometimes provided to the decoder instead of the predicted word.

3.4.4 Training Configuration

The model was trained for 20 epochs using the Adam optimizer (learning rate = 0.001). Loss function was Cross-Entropy Loss with padding tokens ignored. The version with lowest validation loss was saved.

3.5 Transformer-Based Fine-Tuning

For the Latin Darija translation model, we selected `Helsinki-NLP/opus-mt-mul-en`, a multilingual sequence-to-sequence model designed for translation from multiple languages to English. For the Arabic-script Darija dataset, we used `Helsinki-NLP/opus-mt-ar-en`, specifically optimized for Arabic-to-English translation.

3.5.1 Tokenization

Pretrained tokenizers associated with each model were applied. Both source and target sentences were converted to numerical token representations. Padding and truncation were applied with a maximum sequence length of 128 tokens.

3.5.2 Training Configuration

The following hyperparameters were defined using Hugging Face's TrainingArguments class:

Table 1: Training Hyperparameters for Transformer Models

Parameter	Value
Output directory	/content/latin_model
Evaluation strategy	epoch
Save strategy	epoch
Learning rate	3e-5
Train batch size (per device)	4
Eval batch size (per device)	4
Number of epochs	3
Weight decay	0.01
FP16 (mixed precision)	True
Logging steps	20
Save total limit	2

Source: Authors' experimental setup

3.5.3 Training Workflow

The Trainer API from Hugging Face managed the complete training workflow. A custom TranslationLogger callback was implemented to display sample translations after each evaluation phase, enabling qualitative monitoring of translation quality.

3.6 Platform Architecture

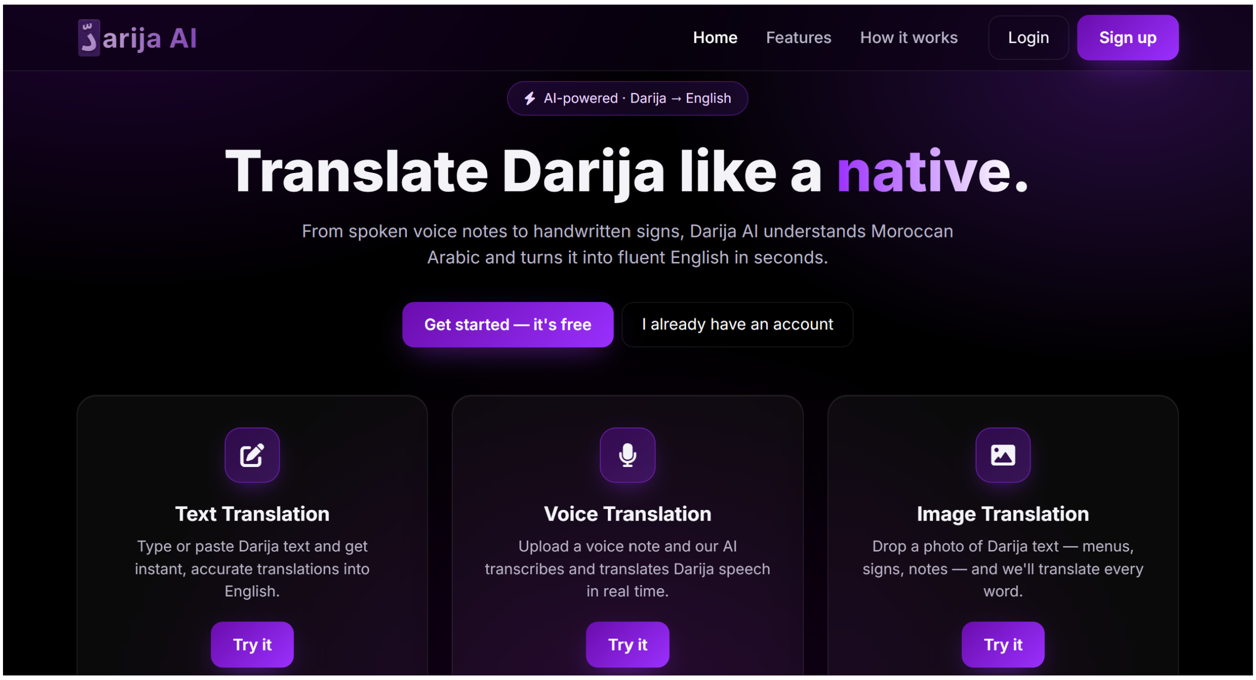

Figure 5: Home page of the translation platform

The Darija AI Translation Platform is a web-based application designed to translate Moroccan Darija into English using artificial intelligence. The system was developed to support multiple input formats, including text, voice, and images. The goal of the project is to make translation more accessible and practical for everyday communication by combining natural language processing, speech recognition, and optical character recognition in a single unified platform.

The project is built using Flask, a lightweight Python web framework that handles routing, user authentication, and API communication. The backend integrates several AI and machine learning technologies, including PyTorch and Hugging Face Transformers for translation, OpenAI Whisper for speech recognition, and EasyOCR for extracting text from images. The system uses MySQL as the database, managed through SQLAlchemy ORM. Additional libraries such as NumPy, Pandas, and SciPy are used for data processing and support tasks.

The system follows a layered architecture composed of three main parts:

Table 2: System Architecture Layers

Layer	**Components**
Frontend	User interface for text, voice, and image input
Backend	Flask routing, authentication, API communication
AI Processing	Translation (Transformers), Speech recognition (Whisper), OCR (EasyOCR)
Database	MySQL with SQLAlchemy ORM for user data and history

3.6.1 Authentication System

The platform includes a secure authentication system that allows users to register, log in, and maintain sessions throughout their usage. Once authenticated, the user's session is stored securely, enabling access to protected routes such as translation and history pages. If a user is not logged in, they are automatically redirected to the login page. This ensures that each user has a personalized and secure experience within the platform.

3.6.2 Translation Pipeline

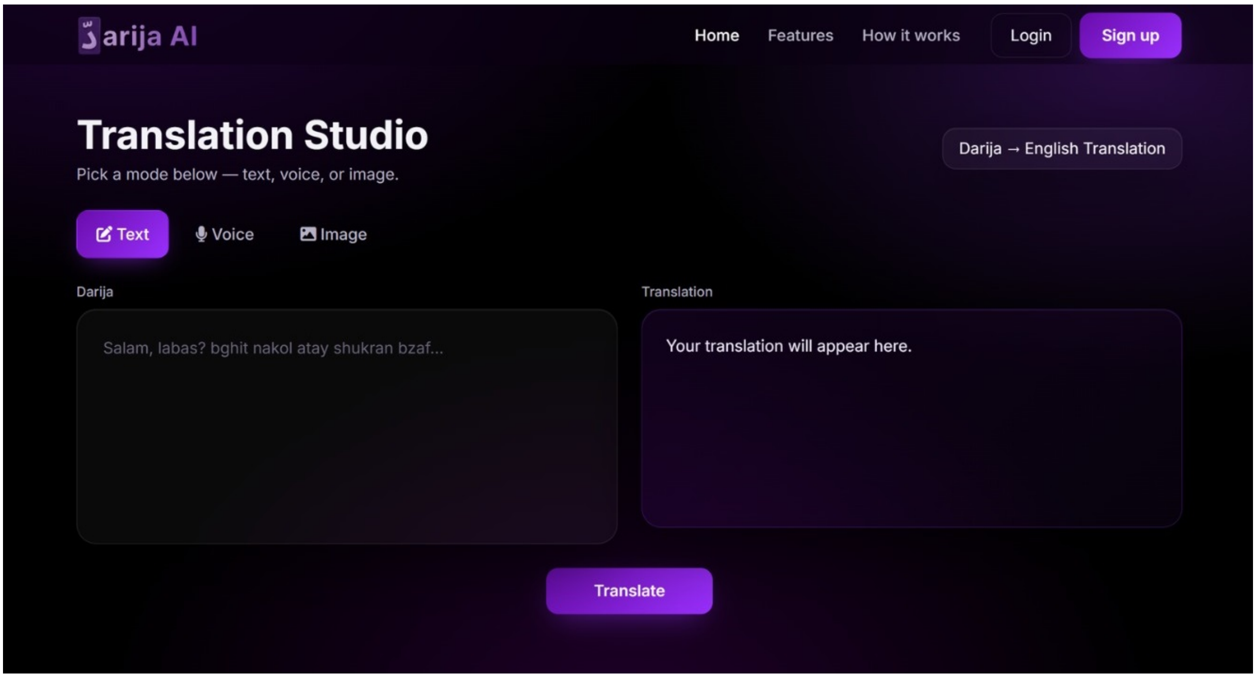

Figure 6: Translation page with input options

The core feature of the platform is the translation engine. The system supports two types of language input detection: Arabic script and Latin script. Based on this detection, the appropriate transformer model is used to generate the English translation.

To improve performance and handle long inputs efficiently, the text is split into smaller chunks before processing. Each chunk is translated separately and then combined to produce the final output. This ensures better accuracy and prevents memory issues when handling large inputs.

3.6.3 Speech Recognition Module

The platform includes a voice input feature that allows users to speak instead of typing. Audio files are processed using the Whisper speech recognition model, which converts spoken language into text. Once transcribed, the resulting text is passed through the translation system. This feature allows users to interact with the system more naturally and is especially useful for mobile or hands-free usage.

3.6.4 Image OCR Module

The system also supports image-based translation. Users can upload images containing text, which are processed using EasyOCR. The OCR engine extracts text from the image, even if the text is handwritten or slightly distorted. Once extracted, the text is sent to the translation model. This feature is useful for translating signs, documents, or screenshots containing Darija text

Every translation performed by the user is stored in a database. The history system keeps track of the original input, translated output, input type (text, audio, or image), and timestamp. This allows users to revisit their previous translations at any time. Each user has a personal history, ensuring data privacy and personalization.

4 Results

4.1 LSTM Model Results

4.1.1 Latin Darija LSTM

The Latin Darija LSTM model showed consistent training loss decrease from 4.75 to 1.68 over 20 epochs. However, validation loss began increasing after epoch 4-5, reaching 5.32 at the final epoch. This divergence indicates overfitting. Final test loss was 5.2565.

4.1.2 Arabic Darija LSTM

The Arabic Darija LSTM showed training loss decrease from 4.70 to 2.18. Validation loss fluctuated between 4.78 and 5.25 without significant improvement. Final test loss was 5.2527.

4.2 Transformer Model Results

4.2.1 Latin Darija Transformer

The fine-tuned transformer model demonstrated successful learning with steady loss reduction across three epochs:

Table 3: Training and Validation Loss per Epoch - Latin Darija Transformer

Epoch	Training Loss	Validation Loss
1	0.0847	0.0695
2	0.0718	0.0621
3	0.0464	0.0604

The final overall training loss was approximately 0.0659. Training time was 5231 seconds (approximately 1 hour 27 minutes) with processing speeds of 40.9 samples per second.

4.2.2 Sample Translation Output

The model was tested on the Darija sentence:

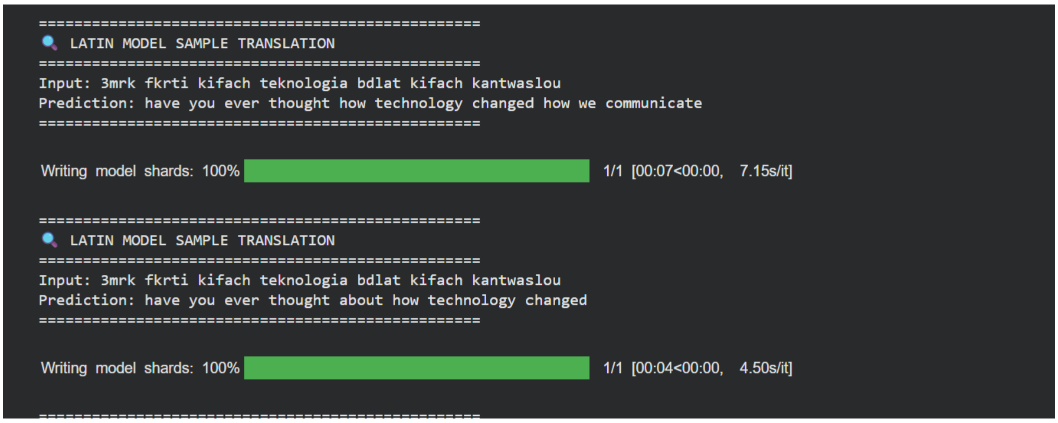

Figure 7: Translation model testing example

These outputs demonstrate successful semantic capture and grammatically coherent English generation.

4.3 Model Comparison

Table 4: Comparison of LSTM and Transformer Models

Metric	**LSTM (Latin)**	**Transformer (Latin)**
Final Training Loss	1.68	0.0464
Final Validation Loss	5.32	0.0604
Test Loss	5.2565	Not reported
Overfitting	Severe	Minimal
Translation Quality	Limited	Good

The transformer architecture significantly outperformed the LSTM baseline model, achieving much lower loss values and generating fluent, context-aware translations.

5 Model Evaluation

To evaluate the performance of the fine-tuned transformer model for Darija-to-English translation, we used standard machine translation metrics on a small test set containing common conversational Darija sentences.

5.1 Evaluation Metrics

Two evaluation metrics were used:

- **BLEU Score**: Measures similarity between generated translations and reference translations using n-gram matching.
- **METEOR Score**: Evaluates translation quality while considering semantic similarity, synonyms, and word order.

5.2 Sample Translation Results

Table 5: Sample Translation Results

Darija Input	**Reference**	**Prediction**
ana fiya jou3	I am hungry	I am hungry
wach nta labas	Are you okay	Are you okay
ghadi nmchi lsou9 ghda	I will go to the market tomorrow	I will go to the market tomorrow
had film zwin bzaf	This movie is very good	This film is very good
ch7al taman	How much is the price	How much does it cost

The model generated accurate translations in most cases and was able to preserve the overall meaning even when different wording was used.

5.3 Evaluation Scores

Table 6: Evaluation Results

Metric	**Score**
BLEU Score	23.22
METEOR Score	0.824

The BLEU score indicates moderate lexical similarity between predictions and references, while the high METEOR score shows strong semantic understanding and meaning preservation.

5.4 Discussion

The results show that the transformer model performs well for Darija-to-English translation despite the limited resources available for Moroccan Darija. In several examples, the model produced translations with wording different from the reference but still semantically correct.

Although the evaluation set was relatively small, the results demonstrate that the model is capable of generating understandable and meaningful English translations from Darija text.

6 Discussion

6.1 Interpretation of Results

The superior performance of transformer models over LSTM can be attributed to the self-attention mechanism, which captures long-range dependencies more effectively than recurrent architectures. For dialectal languages like Darija, where word order can vary significantly and context is crucial for meaning, this advantage is particularly important.

Fine-tuning played a key role in adapting pretrained models to Darija. The multilingual model (opus-mt-mul-en) proved suitable for Latin-script Darija, which contains mixed influences from Arabic transliteration, French, and informal spellings. The Arabic-specific model (opus-mt-ar-en) was appropriate for Arabic-script Darija.

6.2 Limitations

Despite successful results, several limitations remain:

1. Dataset size remains modest compared to major language pairs
2. Spelling variability in Darija continues to challenge consistent tokenization
3. The LSTM model's overfitting indicates model capacity may be insufficient or regularization inadequate
4. Real-time performance on CPU-only deployments is very slow

6.3 Comparison with Literature

Our results align with established findings that transformer architectures outperform RNN/LSTM baselines for machine translation. For dialectal Arabic specifically, our work demonstrates that fine-tuning multilingual or Arabic-specific pretrained models is an effective strategy when limited parallel data is available.

6.4 Implications

This work has practical implications for low-resource dialect translation. The complete platform demonstrates that state-of-the-art translation can be made accessible to non-technical users through multi-modal interfaces. The comparative analysis provides guidance for future researchers on architecture selection for similar tasks.

6.5 Future Work

Further development should include:

1. Expansion of the training dataset through additional crowdsourcing

2. Implementation of on-device inference for offline translation

3. Addition of Darija-to-French and Darija-to-Spanish translation models

4. Deployment as a mobile application

Conclusion

We presented a complete multi-modal platform for Moroccan Darija to English translation. The system integrates fine-tuned transformer-based models for both Latin-script and Arabic-script Darija, achieving low validation loss and fluent translations. A comparative LSTM baseline confirmed the superiority of transformer architectures for this task. The platform includes voice input via Whisper, image OCR via EasyOCR, user authentication, and translation history management.

Our key contributions include: (1) the first dedicated transformer-based translation system for Moroccan Darija handling both scripts, (2) a comprehensive dataset preparation pipeline integrating MADAR and DODa, (3) a practical full-stack application making AI translation accessible to non-technical users.

The increasing validation loss in LSTM models and the success of fine-tuned transformers highlight the importance of architecture choice for dialectal translation tasks. Future work will focus on dataset expansion, mobile deployment, and support for additional target languages.

Acknowledgments

We would like to thank Prof. Cherradi Mohammed for his supervision and guidance throughout this project. We also thank the contributors of the MADAR and DODa datasets for making their work publicly available. This work was supported by the National School of Applied Sciences of Al Hoceima, Abdelmalek Essaadi University.

References

[1] Darija Open Dataset. (2024). *GitHub Repository*. https://github.com/darija-open-dataset/dataset

[2] GeeksforGeeks. (2026). *Visualizing Text Data: Techniques and Applications*. https://www.geeksforgeeks.org/data-visualization/visualizing-text-data-techniques-and-applications/

[3] GeeksforGeeks. (2026). *Text Preprocessing in NLP*. https://www.geeksforgeeks.org/nlp/text-preprocessing-for-nlp-tasks/

[4] Khatabi, A. (n.d.). *Arabic Word Clouds: Text Processing & Rendering*. Kaggle. https://www.kaggle.com/code/amjadkhatabi/arabic-word-clouds-text-processing-rendering

[5] Brownlee, J. (2017). *Building a Plain Seq2Seq Model for Language Translation*. Machine Learning Mastery. https://machinelearningmastery.com/building-a-plain-seq2seq-model-for-language-translation/

[6] Helsinki-NLP. (2020). *opus-mt-mul-en (Version 1.0) [Transformer model]*. Hugging Face. https://huggingface.co/Helsinki-NLP/opus-mt-mul-en